

Capitolo 29 — Processo vs Protocollo

Pubblico	Lettori tecnici e non tecnici del WCM Process & Memory Book
Versione	FROZEN-29
Data master	2026-08-30
Stato	FROZEN
Source path	wcm/documentation/process-memory-book/ chapters/29_processo_vs_protocollo.md
Source SHA	8b39074
Release	2026-08-31T22:48:24.896Z

GitHub main è la source of truth. Questo PDF è una release derivata: non costituisce approvazione né autorità.

PARTE VII — Il Libro dei Protocolli WCM Stato: FROZEN **Data:** 2026-08-30 **Scope:** WCM generale, domain-agnostic

29.0 Due parole simili, due funzioni diverse

Nel linguaggio quotidiano, parole come *processo*, *procedura*, *protocollo*, *regola* e *workflow* vengono spesso usate quasi come sinonimi.

In un sistema operativo complesso, questa ambiguità diventa pericolosa.

Se non è chiaro che cosa descrive il percorso di un'attività e che cosa, invece, impone le condizioni che devono essere rispettate durante quel percorso, diventa difficile capire:

- cosa deve accadere;
- in quale ordine;
- quali controlli sono obbligatori;
- quando fermarsi;
- quali regole valgono soltanto in certe condizioni;
- perché una stessa regola può comparire in attività diverse.

Nel WCM la distinzione di base è semplice:

```
PROCESSO
= come si svolge un'attività

PROTOCOLLO
= cosa deve essere rispettato durante l'esecuzione
```

Questa formula non significa che un processo sia privo di regole, né che un protocollo sia privo di una sequenza.

Significa che i due oggetti hanno una **funzione primaria diversa**.

Il processo organizza il movimento del lavoro.

Il protocollo protegge quel movimento con vincoli, controlli e gate quando sono applicabili.

29.1 Il problema che questa distinzione risolve

Immaginiamo un'attività generica: ricevere una richiesta, preparare un risultato e consegnarlo.

Possiamo descrivere il percorso così:

```
RICHIESTA
→ PREPARAZIONE
→ VERIFICA
→ CONSEGNA
→ CHIUSURA
```

Questa è una descrizione del **flusso**.

Ora immaginiamo che durante questo percorso esistano alcune condizioni obbligatorie:

- prima di modificare qualcosa bisogna assicurarsi di operare nel perimetro corretto;
- prima di dichiarare il lavoro concluso bisogna verificare il risultato;
- se esiste già un'esecuzione equivalente non bisogna crearne una seconda;
- se manca una prova necessaria bisogna fermarsi invece di indovinare.

Queste condizioni non descrivono necessariamente l'intero viaggio dall'inizio alla fine.

Descrivono **come quel viaggio deve essere condotto in sicurezza e coerenza**.

È qui che nasce la distinzione tra processo e protocollo.

Senza questa separazione, ogni flusso dovrebbe ripetere tutte le regole trasversali che possono riguardarlo. Il risultato sarebbe una moltiplicazione di copie, con un rischio crescente di divergenza.

All'estremo opposto, se avessimo soltanto protocolli, sapremmo quali regole rispettare ma non avremmo necessariamente una rappresentazione chiara del ciclo operativo complessivo.

Il WCM mantiene quindi entrambi.

29.2 La definizione corrente nel Process & Protocol Book

La baseline corrente del WCM distingue tre tipi principali di documento operativo nel Process & Protocol Book:

```
PROCESS
→ flusso operativo normale e riutilizzabile

PROTOCOL
→ regole, controlli o gate obbligatori

PLAYBOOK
→ risposta pratica a una situazione o anomalia ricorrente
```

Questo capitolo si concentra sui primi due.

La differenza può essere letta attraverso le domande a cui rispondono.

Processo

| *Come dovrebbe svolgersi questa attività?*

Un processo tende quindi a rendere espliciti:

- il punto di ingresso;
- il trigger;
- gli input;
- gli stati o le fasi;
- la sequenza operativa;
- i gate incontrati lungo il percorso;
- gli output;
- le condizioni di chiusura;
- i failure mode;
- le relazioni con altre procedure.

Protocollo

| *Che cosa deve essere rispettato mentre questa attività viene eseguita?*

Un protocollo tende quindi a rendere espliciti:

- il perimetro in cui si applica;
- il trigger che lo rende rilevante;
- i controlli obbligatori;
- le condizioni di PASS o BLOCK;
- i divieti o le invarianti da preservare;
- le evidenze necessarie;
- le escalation o failure condition pertinenti.

La parola chiave è **tende**.

Non si tratta di due stampi rigidi e reciprocamente esclusivi. La classificazione riguarda la responsabilità principale del documento, non la presenza o assenza assoluta di singole sezioni.

29.3 Un'analogia semplice: percorso e regole di circolazione

Un'analogia pedagogica può aiutare.

Supponiamo di dover andare da un punto A a un punto B.

Il **processo** assomiglia al percorso:

```
PARTENZA
→ TRATTO 1
→ INCROCIO
→ TRATTO 2
→ ARRIVO
```

Il **protocollo** assomiglia alle regole che devono essere rispettate mentre percorriamo quella strada:

```
SEMAFORO ROSSO → STOP  
PRECEDENZA → VERIFICA  
STRADA CHIUSA → NON PROSEGUIRE
```

La strada e le regole non competono tra loro.

Servono a cose diverse.

Il percorso senza regole può portare alla destinazione in modo incoerente o rischioso.

Le regole senza percorso non dicono, da sole, quale viaggio dobbiamo compiere.

L'esempio è puramente pedagogico. Non definisce nuove regole WCM e non implica che ogni processo o protocollo abbia una struttura identica a quella di un sistema stradale.

29.4 Il processo risponde soprattutto alla domanda “come avanza il lavoro?”

Un processo WCM descrive un flusso operativo normale e riutilizzabile.

La parola **flusso** è importante.

Il processo rende visibile il passaggio da una condizione a quella successiva.

Un esempio canonico della struttura si vede in **PROC-001 – Service Job Lifecycle**, che rappresenta un ciclo di vita attraverso stati persistenti e transizioni.

In forma semplificata:

```
HOLD  
→ READY  
→ IN_PROGRESS  
→ DONE
```

Accanto al flusso, il processo dichiara trigger, regole, output di chiusura e failure mode.

Questo mostra una cosa importante: **processo non significa “sequenza priva di regole”**.

Le regole possono essere necessarie per definire correttamente le transizioni del processo stesso.

Il punto è un altro: la funzione primaria del documento resta governare il ciclo operativo dell'attività.

Il processo ci permette di rispondere a domande come:

- dove siamo nel ciclo?
- quale transizione è possibile adesso?
- che cosa deve accadere prima della fase successiva?
- quando l'attività è realmente conclusa?
- quale stato rappresenta un blocco o una deviazione?

29.5 Il protocollo risponde soprattutto alla domanda “quali condizioni devo rispettare?”

Un protocollo mette al centro una disciplina obbligatoria.

Non nasce necessariamente per raccontare l'intero ciclo di vita di un'attività. Nasce per impedire che una determinata classe di operazioni venga eseguita senza i controlli previsti.

PROT-001 – Git & Working Tree Safety, per esempio, ha uno scopo preciso: evitare che un'operazione lavori sopra uno stato ambiguo, perda lavoro esistente o aggiri un blocco mediante azioni distruttive.

Il protocollo contiene:

- un gate iniziale;
- controlli da eseguire;
- regole obbligatorie;
- condizioni di PASS o BLOCK;
- evidenze che spiegano perché la disciplina è stata introdotta.

Anche qui compare una sequenza.

Questo mostra il caso speculare rispetto al processo: **protocollo non significa “lista statica priva di flusso”**.

Un protocollo può contenere passi ordinati quando servono a eseguire correttamente il controllo.

La sua funzione primaria, però, resta diversa: non descrive il viaggio complessivo; impone le condizioni che devono essere rispettate quando quel tipo di operazione ricorre.

29.6 Differenza di funzione, non di formato

Una delle confusioni più facili consiste nel cercare di distinguere processo e protocollo guardando soltanto la forma del documento.

Per esempio:

❗ *“Se contiene una lista numerata, allora è un processo.”*

Oppure:

❗ *“Se contiene un gate, allora è un protocollo.”*

Questi criteri sarebbero sbagliati.

Un processo può avere gate.

Un protocollo può avere una sequenza.

Entrambi possono avere trigger, owner, input, output, failure mode ed evidenze quando pertinenti.

La distinzione deve essere cercata nella **responsabilità operativa primaria**:

Domanda	Processo	Protocollo
Che cosa organizza soprattutto?	Il ciclo o flusso dell'attività	Le condizioni obbligatorie dell'esecuzione

Domanda principale	Come procede il lavoro?	Cosa deve essere rispettato?
Centro logico	Fasi, stati, transizioni, output	Regole, controlli, gate, invarianti
Può contenere gate?	Sì	Sì
Può contenere una sequenza?	Sì	Sì
Può avere failure mode?	Sì	Sì
È automaticamente applicabile a tutto?	No	No

L'ultima riga è particolarmente importante.

Né un processo né un protocollo diventano universali soltanto perché esistono nel Process Book.

La loro applicabilità dipende dal loro scope, dal trigger e dal contesto operativo previsto dalla baseline corrente.

29.7 Trigger: quando entra in gioco l'uno e quando entra in gioco l'altro

Il **trigger** è il punto in cui una procedura diventa rilevante.

Per un processo, il trigger spesso indica l'inizio o una transizione significativa del ciclo operativo.

Per un protocollo, il trigger indica invece che è comparsa una condizione che richiede l'applicazione di una disciplina specifica.

Prendiamo un esempio astratto e non canonico.

Un processo potrebbe iniziare quando arriva una richiesta di produrre un documento.

Durante quel processo, un protocollo di verifica potrebbe diventare applicabile soltanto quando si entra nella fase di consegna.

```

RICHIESTA
→ PREPARAZIONE
→ REVISIONE
→ [TRIGGER DEL CONTROLLO]
→ CONSEGNA

```

Il protocollo non deve necessariamente essere “acceso” in ogni momento del processo.

Si applica quando ricorrono le condizioni per cui è stato definito.

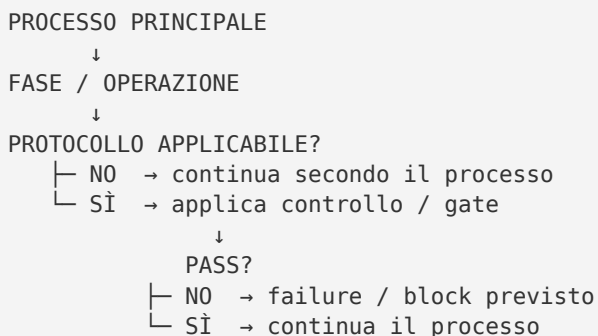
Questo è uno dei motivi per cui il WCM esegue il routing dei protocolli in funzione della richiesta, dell'operazione e dei trigger applicabili, invece di trattare ogni protocollo come una checklist universale da eseguire sempre.

29.8 Un processo può richiamare un protocollo

La relazione tra i due oggetti diventa più chiara osservando come lavorano insieme.

Un processo può dichiarare che, in una certa fase o condizione, deve essere applicato un protocollo.

La struttura concettuale è:



Questa relazione è già visibile nella baseline corrente.

Per esempio, un processo canonico può richiamare esplicitamente un protocollo quando una particolare modalità di esecuzione rende necessario quel controllo.

Il protocollo non sostituisce il processo.

Il processo non assorbe automaticamente il protocollo.

L'uno definisce il percorso operativo; l'altro protegge un punto o una classe di operazioni nel percorso.

29.9 Un protocollo può essere trasversale

Alcune regole riguardano un solo contesto molto specifico.

Altre possono diventare rilevanti in più flussi diversi quando ricorre lo stesso tipo di operazione.

È questa la natura **trasversale** che molti protocolli possono assumere.

La parola “trasversale” non significa però “sempre attivo”.

Significa che la stessa disciplina può essere riutilizzata in più contesti **se il suo trigger e il suo ambito sono pertinenti**.

Un esempio pedagogico:

supponiamo che esista una regola secondo cui una consegna non può essere dichiarata completata senza una verifica dell'esito.

Quella regola potrebbe essere rilevante in processi differenti che producono una consegna, senza dover essere riscritta da zero all'interno di ogni processo.

Nel WCM questo principio evita di duplicare discipline comuni e aiuta a mantenere una sola fonte autorevole per la procedura applicabile.

Ma la trasversalità non deve essere inferita arbitrariamente: deriva dallo scope e dai trigger del protocollo canonico.

29.10 Gate: dove processo e protocollo si incontrano

Il **gate** è un punto in cui non basta dire “continuiamo”.

Serve verificare una condizione.

Un gate può appartenere direttamente alla logica di un processo oppure essere definito da un protocollo applicabile.

In entrambi i casi, la sua funzione è impedire che una transizione avvenga quando manca una condizione richiesta.

```
STATO ATTUALE
→ CONTROLLO
→ PASS?
  └─ SÌ → transizione consentita
  └─ NO → non avanzare secondo il percorso previsto
```

Questa è una zona di sovrapposizione funzionale importante.

Il fatto che entrambi possano contenere gate non annulla la distinzione tra processo e protocollo.

Il gate di un processo può essere parte intrinseca della sua macchina di stato o del suo ciclo.

Il gate di un protocollo protegge invece la conformità a una disciplina che diventa applicabile in quel punto.

Per il lettore, la domanda utile non è quindi:

| *“Chi possiede i gate?”*

ma:

| *“Questo gate serve a governare la progressione del processo o ad applicare una disciplina obbligatoria che protegge l'esecuzione?”*

29.11 Input e output: anche qui la differenza è di ruolo

Un processo riceve input per poter svolgere un'attività e produce output che rappresentano l'avanzamento o il risultato del ciclo.

Un protocollo può anch'esso ricevere input e produrre output, ma spesso questi servono a decidere se l'esecuzione può continuare in modo conforme.

In termini pedagogici:

```
PROCESSO
input → lavoro → output operativo

PROTOCOLLO
condizione/evidence → controllo → PASS / BLOCK / esito previsto
```


Non è una formula universale né uno schema dati obbligatorio.

Serve a mettere a fuoco la funzione.

Un protocollo può produrre evidenze più articolate di un semplice PASS/BLOCK e un processo può includere molte verifiche intermedie. La baseline di ogni elemento canonico prevale sempre sulla semplificazione editoriale.

29.12 Failure mode: due prospettive sul fallimento

Anche i failure mode aiutano a capire la differenza.

Nel processo, una failure tende a significare che il ciclo operativo non può avanzare o chiudersi secondo le condizioni previste.

Nel protocollo, una failure tende a significare che una regola, un controllo o un gate non è stato soddisfatto.

Le due cose possono coincidere operativamente.

Per esempio:

```
PROTOCOLLO → FAIL
      ↓
PROCESSO → NON PUÒ AVANZARE
```

Ma è utile conservare la causa corretta.

Dire soltanto “il processo è bloccato” nasconde il motivo.

Dire “il protocollo applicabile ha fallito perché manca la condizione richiesta” mantiene la spiegazione e rende più chiaro quale parte del sistema deve essere corretta.

Questa separazione aiuta anche a non trasformare ogni problema in una modifica del processo principale.

A volte il processo è corretto: è semplicemente un gate applicabile che non è stato superato.

29.13 Processo, protocollo e authority

Né il processo né il protocollo creano authority dal nulla.

Un documento operativo descrive il comportamento autorizzato della baseline; non può ampliare autonomamente goal, scope, permessi o potere decisionale.

Questa distinzione è essenziale perché una regola tecnicamente applicabile non equivale automaticamente all'autorità di eseguire qualunque azione collegata.

In forma semplice:

```
PROCEDURA APPLICABILE
≠
AUTHORITY ILLIMITATA
```

Il processo dice come operare nel proprio perimetro.

Il protocollo dice quali condizioni rispettare nel proprio perimetro.

L'authority continua a provenire dalle fonti e dai gate competenti previsti dal WCM.

Questo capitolo non modifica tali authority e non introduce nuovi diritti di scrittura, approvazione o chiusura.

29.14 Processo, protocollo e routing

Nei capitoli precedenti abbiamo visto che il WCM non parte caricando indiscriminatamente tutte le procedure disponibili.

La richiesta viene interpretata e il sistema identifica ciò che serve.

In modo semplificato:

```
RICHIESTA
→ INTENTO / GOAL / SCOPE
→ PROCESSO APPLICABILE
→ OPERAZIONI E TRIGGER
→ PROTOCOLLI APPLICABILI
→ EXECUTION
```

Questo ordine concettuale non significa che esista sempre un solo processo né che i protocolli vengano scoperti soltanto dopo l'avvio materiale del lavoro.

Significa che la conoscenza del processo aiuta a capire **che cosa stiamo facendo**, mentre il routing dei protocolli aiuta a capire **quali discipline dobbiamo rispettare per farlo**.

La baseline corrente prevede inoltre protocolli condizionali e relazioni process → protocol: l'applicabilità deve essere determinata dal contesto, non dalla semplice presenza del protocollo nel registro.

29.15 Perché non mettere tutto dentro ogni processo

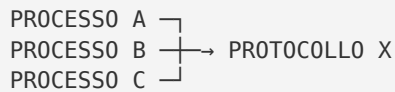
Potremmo immaginare un sistema in cui ogni processo contiene integralmente tutte le regole che potrebbero riguardarlo.

All'inizio sembrerebbe più semplice.

Con il tempo, però, emergerebbero problemi prevedibili:

- la stessa regola copiata in molti documenti;
- versioni diverse della stessa disciplina;
- aggiornamenti applicati in un punto e dimenticati negli altri;
- processi sempre più lunghi;
- difficoltà nel capire quale copia sia quella autorevole.

La separazione tra processo e protocollo permette invece di esprimere una relazione.



Questa rappresentazione è pedagogica: non afferma che ogni protocollo debba essere condiviso da tre processi o che tutte le relazioni abbiano questa forma.

Mostra soltanto il vantaggio architetturale di poter riferire una disciplina comune senza duplicarla.

La baseline corrente del Process Book rafforza infatti il principio di una sola source of truth per ogni procedura.

29.16 Perché non usare soltanto protocolli

Esiste anche il rischio opposto.

Se il sistema contenesse soltanto regole e gate, potremmo sapere perfettamente cosa non fare ma avere una visione frammentaria del lavoro da svolgere.

Una raccolta di protocolli potrebbe dirci:

- verifica questo;
- non duplicare quello;
- fermati se manca questa prova;
- usa quella fonte prima di procedere.

Ma non necessariamente descriverebbe:

DA DOVE PARTE L'ATTIVITÀ
→ COME AVANZA
→ QUALI STATI ATTRAVERSA
→ QUANDO È DAVVERO TERMINATA

È il processo a fornire questa continuità operativa.

Per questo nel WCM processi e protocolli non sono alternative progettuali.

Sono due categorie complementari del medesimo sistema operativo.

29.17 La relazione non è una gerarchia semplice

Un altro errore sarebbe pensare:

| *“Il processo è sempre superiore e il protocollo è sempre un sottolivello.”*

La baseline non definisce questa relazione come una gerarchia universale.

Un protocollo può essere richiamato da un processo, ma può anche proteggere una classe di operazioni che ricorre in più punti del sistema.

Un processo può inoltre richiamare altri processi quando il flusso lo richiede.

È quindi più utile pensare a una rete di relazioni operative che a una piramide rigida.

PROCESSO

- | può richiamare → PROCESSO
- | può richiedere → PROTOCOLLO
- | può incontrare → GATE / FAILURE / OUTPUT

PROTOCOLLO

- | si applica quando scope + trigger lo rendono pertinente

La rappresentazione è descrittiva e pedagogica; le relazioni effettive devono essere ricavate dalle procedure canoniche e dal registro, non inventate a partire dallo schema.

29.18 Come leggere il registro senza confondersi

Il punto di ingresso operativo corrente del Process & Protocol Book è [PROCESS_REGISTER.md](#).

Nonostante il nome, il registro indicizza sia i **12 processi** canonici correnti sia i **20 protocolli** canonici correnti, oltre al playbook e ai template pertinenti.

Nella struttura corrente non esiste un secondo file separato chiamato [PROTOCOL_REGISTER.md](#).

Questo è un dettaglio implementativo importante per evitare una falsa inferenza durante la navigazione.

La struttura attuale è:

```
wcm/process-book/  
├── README.md  
├── PROCESS_REGISTER.md  
├── processes/  
├── protocols/  
├── playbooks/  
└── templates/
```

Il nome del registro non cambia però la distinzione semantica tra le due categorie.

All'interno dello stesso indice esistono sezioni distinte per processi e protocolli, ciascuna con ID stabile, titolo, stato e scopo.

29.19 Maturity: non esiste una maturità unica di “tutti i processi” o “tutti i protocolli”

Il registro corrente mostra una cosa che deve essere preservata anche nel linguaggio editoriale: gli elementi non hanno tutti lo stesso livello di maturità.

Alcuni sono **VALIDATED**.

Altri sono **ACTIVE** con field validation in corso, prima validazione sul campo o qualificazioni più specifiche.

Di conseguenza non sarebbe corretto dire:

| *“I processi WCM sono tutti validati.”*

oppure:

| *“I protocolli WCM sono tutti field validated.”*

La categoria documentale è baseline corrente.

La **maturity appartiene al singolo PROC o PROT**, nel perimetro dichiarato dalla sua fonte canonica e dal registro.

Questo capitolo spiega la distinzione architetturale tra le categorie; non promuove né modifica la maturità di alcuna procedura.

29.20 Un esempio pedagogico completo

Consideriamo una richiesta astratta:

| *“Prepara una consegna sulla base di alcune fonti e chiudi il lavoro quando il risultato è verificato.”*

Un possibile modello pedagogico — non una nuova procedura WCM — potrebbe essere:

```
PROCESSO
RICHIESTA
→ RACCOLTA INPUT
→ PREPARAZIONE
→ VERIFICA
→ CONSEGNA
→ CHIUSURA
```

Durante questo percorso potrebbero emergere controlli diversi:

```
PROTOCOLLO A
se devi modificare uno stato persistente
→ verifica target e condizioni di sicurezza

PROTOCOLLO B
se stai per dichiarare la chiusura
→ verifica acceptance/evidence

PROTOCOLLO C
se esiste già un'esecuzione equivalente
→ evita duplicazione
```

Il processo ci dice **dove siamo e dove dobbiamo andare**.

I protocolli ci dicono **quali condizioni dobbiamo rispettare nei punti in cui diventano applicabili**.

Se il protocollo B fallisce, il processo non può essere dichiarato chiuso soltanto perché la consegna materiale esiste.

Se il protocollo C non è applicabile, non deve essere eseguito come rituale universale.

L'esempio serve soltanto a rendere intuitiva la relazione e non definisce nuovi ID, trigger o gate WCM.

29.21 Le confusioni da evitare

Possiamo ora riassumere le principali confusioni.

“Un processo è una sequenza; un protocollo non lo è.”

Falso.

Un protocollo può avere una sequenza di controllo.

“Un protocollo ha gate; un processo no.”

Falso.

I processi possono contenere gate necessari alle loro transizioni.

“Ogni protocollo vale sempre.”

Falso.

Scope e trigger determinano l'applicabilità.

“Se un protocollo è applicabile, allora crea authority.”

Falso.

Applicabilità e authority sono concetti distinti.

“Il registro dei processi contiene soltanto processi.”

Falso nella struttura corrente.

`PROCESS_REGISTER.md` indicizza processi, protocolli e gli altri elementi operativi previsti.

“Processo e protocollo sono due nomi per la stessa cosa.”

Falso.

Possono condividere sezioni e strutture, ma hanno responsabilità operative primarie differenti.

29.22 La regola finale da ricordare

La distinzione più utile può essere compressa in due domande:

PROCESSO

→ COME AVANZA IL LAVORO?

PROTOCOLLO

→ COSA DEVE ESSERE RISPETTATO MENTRE AVANZA?

Il processo dà continuità al lavoro.

Il protocollo gli dà disciplina.

Il processo senza protocolli applicabili rischia di avanzare senza i controlli necessari.

Il protocollo senza un processo o un'operazione a cui applicarsi resta una regola senza il contesto operativo che la rende rilevante.

Nel WCM i due elementi lavorano quindi insieme, ma non vengono confusi: la baseline mantiene separati il **flusso operativo riutilizzabile** e le **regole, i controlli o i gate obbligatori** che devono essere rispettati quando il loro scope e trigger diventano pertinenti.

Source Map

Fonti canoniche / baseline primaria

- [wcm/process-book/README.md](#) — definizione corrente di PROCESS, PROTOCOL e PLAYBOOK; struttura e principi di manutenzione del Process & Protocol Book;
- [wcm/process-book/PROCESS_REGISTER.md](#) — indice operativo corrente di processi e protocolli, ID, stati, scopi e maturity qualifiers;
- [wcm/documentation/process-memory-book/BOOK_INDEX.md](#) — mapping editoriale canonico di CH29 e collocazione nella PARTE VII.

Fonti di anatomia usate come esempi tecnici

- [wcm/process-book/processes/PROC-001_SERVICE_JOB_LIFECYCLE.md](#) — esempio canonico di processo con flusso, trigger, regole, output e failure mode;
- [wcm/process-book/protocols/PROT-001_GIT_WORKTREE_SAFETY.md](#) — esempio canonico di protocollo con gate, regole obbligatorie ed esito del controllo.

Qualificatori di verità e maturity

- la distinzione PROCESS / PROTOCOL descritta in questo capitolo è parte della baseline corrente del Process & Protocol Book;
- la maturity non viene generalizzata per categoria: ogni **PROC-*** e **PROT-*** conserva il proprio stato dichiarato nel registro e nella fonte canonica;
- nessun claim di FIELD VALIDATION universale;
- nessun claim di originalità assoluta;
- nessuna nuova regola, gerarchia, relazione o trigger WCM introdotti dal capitolo;
- gli esempi non canonici sono esclusivamente pedagogici e domain-agnostic;
- in caso di divergenza tra una semplificazione editoriale e la procedura canonica, prevale la procedura canonica corrente.